

Troll Farm — best bot v1.4.5-seedreserve

A reading guide to the arena's strongest bot, written for a developer fluent in **C++**, **Go** and **Python** who is new to **Rust**.

CodinGame Spring Challenge 2026 · "Troll Farm" · 2-player, ~300 turns, 50 ms/turn · the live decider is `fn decide_elite(state: &State) -> Vec<String>`.

How to read this document. **Part 0** is a two-minute setup: how to compile and run a single Rust file on this machine with `emacs + make`, so you can try anything below immediately. The rest is three reading parts. **Part A** is a one-time Rust reference: every language construct that appears in the bot, explained in one or two sentences against the C++/Go/Python you already know — read it once, then refer back. **Part B** is the game and the strategy: *what* the bot does and *why*. **Part C** walks the actual source top to bottom, block by block, with short verbatim excerpts, pointing back to Part A for the syntax and Part B for the intent. The readable source is `docs/best-bot-v1.4.5.rs` (the arena submission with all dead code removed; it still compiles with `rustc --edition 2021 -O`).

Part 0 — Running Rust on this machine (emacs + make)

To try the constructs in Part A you do **not** need `cargo` or a project. A single `.rs` file with a `fn main()` is a complete program: `rustc` compiles it to one binary. A two-line `Makefile` wraps that so the whole loop is **edit** → **make** → **read output**. This machine has `rustc 1.75.0` and GNU `make` already installed — nothing else is required.

The scratch pad (already set up)

Two files live in `rust-scratch/`: `scratch.rs` (a runnable file whose examples mirror Part A — `&str/String`, `Vec/HashMap`, iterators/closures, `Option/match/if let`, a `struct + impl`) and `Makefile`. Edit `scratch.rs`, replace the examples with whatever you want to test, and run one command:

```
# from rust-scratch/ :
make          # compile scratch.rs (edition 2021) and run it — prints its output
make FILE=x.rs # compile + run some other single file instead
make check    # type-check only (fast) — no binary produced, no run
make clean    # delete the built binary (/tmp/rust-scratch-bin)
```

The whole `Makefile` is just a wrapper around `rustc` — no magic, portable to any Rust box:

```
FILE ?= scratch.rs
BIN  := /tmp/rust-scratch-bin
run:
    @rustc --edition 2021 $(FILE) -o $(BIN) && ./$(BIN) # compile then run
```

Without `make` at all, the same thing is one line: `rustc --edition 2021 scratch.rs -o /tmp/s && /tmp/s`. Add `-O` for an optimized build (as the arena uses); omit it for faster compiles while iterating.

The emacs loop. Open `scratch.rs`, then `M-x compile RET make RET`. Output (or compiler errors) appear in the `*compilation*` buffer; **compiler errors are clickable** — press `C-x `` (or click) to jump straight to the offending line, since `rustc`'s `file:line:col` messages are exactly what emacs' error parser understands. After the first run, `M-x recompile` (or bind it) re-runs the same command with no prompts. To make `make` the default so you never retype it, add `(setq compile-command "make")` to your config (or set it per-file). Base emacs needs no `rust-mode` for this — it treats `.rs` as text and `make / rustc` do the work (if you want syntax highlighting later, `M-x package-install rust-mode`).

Why edition 2021: the arena bot and the examples use 2021 idioms (notably inline format args like `println!("{}", x)`). Plain `rustc scratch.rs` defaults to edition 2015 and would reject those — hence the flag is baked into the Makefile.

Part A — Reading Rust as a C++/Go/Python developer

Rust will feel familiar in the small (C-like syntax, methods, generics) but has three ideas that have no direct analogue in the languages you know: **ownership & borrowing** (who may read or write a value, enforced at compile time), **no null / no exceptions** (absence is `Option`, failure is `Result`), and **explicit copying** (nothing heap-allocated is duplicated unless you say `.clone()`). Everything below is a construct that literally appears in this bot.

A1. The three hard ideas, once

Ownership. Every value has exactly one owner; when the owner goes out of scope the value is freed (RAII, like C++ destructors, but the compiler tracks it). Assigning or passing a non-trivial value *moves* ownership — the source name becomes unusable — unless the type is `Copy` (small stack types: integers, `char`, tuples of those) or you explicitly `.clone()` it. There is no garbage collector.

Borrowing. Instead of moving, you lend a reference. A **shared borrow** `&T` is read-only and you may have any number at once; an **exclusive borrow** `&mut T` is read-write and you may have exactly one, with no `&T` alive at the same time. The compiler's "borrow checker" rejects code that violates this. The pay-off: a value is either *shared and immutable* or *uniquely mutable*, which is how Rust prevents data races and dangling pointers with zero runtime cost. In this bot, `decide_elite(state: &State)` only ever reads the world; the parser takes `reader: &mut impl BufRead` because reading a line mutates the stream position.

Absence, not null. A maybe-missing value is `Option<T> = Some(x) or None`; there is no null pointer. A maybe-failing call is `Result<T,E>`. You must handle both cases, which is why you will see `if let Some(x) = ...`, `.unwrap_or(default)`, and the `?` operator everywhere.

A2. Types, memory & copying

Construct	What it is (vs C++ / Go / Python)
<code>&str</code>	Borrowed string slice : an immutable view into text you do not own — essentially a C++ <code>std::string_view</code> or a read-only Go <code>string</code> . Cheap to pass; no allocation. Seen as VERSION: <code>&str</code> and every <code>t: &str</code> parameter.
<code>String</code>	Owned, growable, heap UTF-8 string — C++ <code>std::string</code> , Python <code>str</code> . <code>Tree.tree_type</code> owns its text; <code>format!(...)</code> produces a <code>String</code> .
<code>&T</code> / <code>&mut T</code>	Shared (read-only, many) vs exclusive (read-write, one) borrow — see A1. <code>&State</code> , <code>&HashSet<Cell></code> , <code>&mut String</code> .
<code>.clone()</code>	Explicit deep copy. Rust never copies heap data implicitly (unlike a C++ copy-constructor or a Python/Go reference assignment); you ask for it, so the cost is visible. <code>state.walkable.clone()</code> , <code>my_trolls.clone()</code> .
<code>as cast</code>	Explicit numeric conversion; Rust never converts integer types implicitly. <code>my.len() as i32</code> , <code>x as i8</code> .

A3. Collections & compound types

Construct	Meaning
<code>Vec<T></code>	Growable array — C++ <code>std::vector</code> , Go slice, Python <code>list</code> . <code>Vec<String></code> (the turn's commands), <code>Vec<Troll></code> .
<code>HashMap<K,V></code>	Hash map / dict — <code>unordered_map</code> , Go <code>map</code> , Python <code>dict</code> . <code>HashMap<Cell,i32></code> is a BFS distance-per-cell.
indexing <code>dist[&(x, y)]</code>	Read a <code>HashMap</code> by key with the <code>[]</code> operator. Two things to notice. (1) The <code>&</code>: map indexing borrows the key — the signature is <code>fn index(&self, key: &K) -> &V</code> , so you pass <code>&(x, y)</code> (a reference to the coordinate tuple), <i>not</i> the tuple by value. (Contrast <code>Vec: v[i]</code> takes <code>usize</code> by value.) The result is <code>&V</code> , but because the value here is <code>i32</code> (Copy) <code>let d = dist[&(x, y)]</code> just copies the number out. (2) It panics if the key is absent — like Python's <code>dict[k]</code> raising <code>KeyError</code> , <i>not</i> Go's silent zero-value. So <code>[]</code> is used only where the key is guaranteed present (here <code>(x, y)</code> was just <code>pop_front()</code> ed from the BFS queue); everywhere a cell might be missing the bot uses <code>.get(&key) -> Option</code> instead (see <code>d.get(c).copied().unwrap_or(1<<30)</code>).
<code>HashSet<T></code>	Set of unique keys. <code>walkable</code> , <code>reserved</code> , and <code>seed_cells</code> are all <code>HashSet<Cell></code> .
<code>VecDeque<T></code>	Double-ended queue; the BFS frontier (<code>push_back</code> / <code>pop_front</code>).
<code>[T; N]</code>	Fixed-size stack array (C array / <code>std::array</code>). <code>[i32; 6]</code> inventory, <code>[Cell; 4]</code> neighbours, <code>[bool; 3]</code> funding flags.
tuple <code>(A,B)</code> · <code>.0</code> / <code>.1</code>	Anonymous fixed group — C++ <code>std::tuple</code> , Python tuple. Fields are positional: <code>c.0</code> , <code>c.1</code> ; <code>GE_SPEC.0</code> is movement-speed. <code>type Cell = (i32,i32)</code> names the coordinate pair.
<code>type X = ...</code>	Type alias — C++ <code>using</code> / <code>typedef</code> . <code>type Cell = (i32, i32)</code> .
<code>struct</code> · <code>.field</code>	Named-field record — C++/Go <code>struct</code> . <code>Troll { id, x, y, ... }</code> , read as <code>u.id</code> .

A4. Option, matching & control flow

Construct	Meaning
<code>Option<T>, Some / None</code>	Maybe-present value — C++ <code>std::optional</code> , Go's comma-ok, replacing null. <code>d.get(&cell)</code> and <code>.min_by_key(...)</code> return <code>Option</code> .
<code>if let Some(x) = e { ... }</code>	Bind <code>x</code> and run the block only when <code>e</code> is <code>Some</code> — a concise "unwrap if present".
<code>match</code>	Exhaustive multi-way branch / pattern match — a <code>switch</code> that must cover all cases. <code>match ch { '0' => ..., '.' => ..., _ => {} };</code> <code>_</code> is the catch-all.
<code>while let Some(x) = it { ... }</code>	Loop while the pattern matches; the BFS drains its queue until <code>pop_front()</code> yields <code>None</code> .
<code>for &cell in sources</code>	The <code>&</code> in the pattern dereferences each borrowed item, so <code>cell</code> is a <code>Cell</code> value rather than <code>&Cell</code> .
early <code>return x;</code>	Yes — Rust has a mid-function <code>return</code>, identical to C++/Go/Python. <code>return x;</code> (or bare <code>return;</code> for a <code>()</code> function) exits immediately from anywhere, including inside a <code>match</code> arm or loop. The twist is <i>idiom</i> : a Rust function's normal result is its last expression written with NO semicolon (the "tail expression"), so <code>return</code> is reserved for early exits / guard clauses , not the final value. Compare <code>fell_ok</code> : the guard bails early — <code>if seed_cells.contains(&p.pos()) { return false; }</code> — while the ordinary result is the trailing <code>p.size >= 1</code> (no <code>return</code> , no <code>;</code>). Writing <code>return p.size >= 1;</code> would compile too; it is just less idiomatic. A stray <code>;</code> on that last line would instead make the function return <code>()</code> and fail to type-check — the #1 surprise for newcomers.
<code>? on Option</code>	In a function that returns <code>Option</code> , <code>read_line(reader)?</code> returns <code>None</code> immediately if the call was <code>None</code> , else unwraps it — Go's <code>if err != nil { return }</code> in one character; a specialised <i>early return</i> . (Elsewhere the parser uses <code>.unwrap()</code> , which <i>panics</i> on malformed input — acceptable because the referee guarantees well-formed input.)

A5. Closures & iterator chains (the workhorse)

Most of the bot's logic is expressed as lazy iterator pipelines over the trees, cells and trolls. Read a chain left to right as a data-flow. These are C++20 ranges / Python generator expressions / Go loops, but composable.

Construct	Meaning
closure <code> x expr</code>	Anonymous inline function — C++ lambda, Go func literal, Python <code>lambda</code> . <code> p p.size >= 2</code> ; a block body uses braces.
capturing	A closure automatically borrows the outer variables it names (no C++-style capture list). <code>fell_ok</code> captures <code>seed_cells</code> , <code>liquidation</code> , <code>farm_r</code> ; <code>nearest_fell</code> captures <code>u</code> , <code>d</code> , <code>reserved</code> .
<code>.iter()</code> / <code>.into_iter()</code>	Iterate by borrow (<code>&T</code>) or by value (<code>T</code> , consuming the collection). Lazy.
<code>.filter(pred)</code>	Keep items where the closure is true.
<code>.map(f)</code>	Transform each item.
<code>.filter_map(f)</code>	<code>map</code> + drop the <code>None</code> s in one pass (<code>f</code> returns <code>Option</code>).
<code>.flat_map(f)</code>	Map each item to an iterator and flatten — used to expand each iron cell into its four neighbours.
<code>.min_by_key(x k)</code>	The item with the smallest key; returns <code>Option</code> . This is the core "pick the nearest / cheapest target" primitive.
<code>.any(pred)</code> · <code>.count()</code>	"Does any match" (bool) · "how many match". <code>!my.iter().any(u u.chop_power >= 2)</code> .
<code>.find(pred)</code>	First matching item as <code>Option</code> ("is a troll standing on a tree?").
<code>.sum()</code> · <code>.take(k)</code>	Add up items · keep the first <code>k</code> (the seed reserve is <code>.take(GE_SEED_RESERVE)</code>).
<code>.collect()</code>	Materialise a lazy iterator into a real collection (<code>Vec</code> , <code>HashMap</code> ...); target type is inferred or given by turbofish.
<code>.copied()</code> · <code>.unwrap_or(d)</code> · <code>.map_or(d, f)</code>	<code>&T → T</code> by copy · <code>Option</code> -or-default · transform-or-default. <code>d.get(c).copied().unwrap_or(1<<30)</code> = "its distance, or infinity".
<code>.sort_by_key(x k)</code>	In-place sort by a derived key; tuple keys sort lexicographically , which is how the seed reserve orders bananas by (<code>-size</code> , <code>-fruits</code> , <code>dist</code> , <code>pos</code>).

A6. Macros, traits & interior mutability

Construct	Meaning
<code>macro name!(...)</code>	The <code>!</code> marks a macro (expanded at compile time), not a function. <code>format!</code> builds a <code>String</code> ; <code>writeln!</code> / <code>eprintln!</code> do formatted output; <code>thread_local!</code> declares per-thread statics. Macros take variadic, compile-time-checked format arguments.
<code>trait</code>	A set of methods a type promises to provide — a Go interface / C++ concept. <code>#[derive(Clone)]</code> auto-implements the <code>Clone</code> trait for <code>Troll/Tree</code> ; <code>reader: &mut impl BufRead</code> accepts any type implementing <code>BufRead</code> . (The library twin of this file is written as <code>impl Strategy for GoldElite</code> , where <code>Strategy</code> is exactly such an interface with one <code>decide(...)</code> method.)
<code>impl T { fn m(&self) }</code>	Attach methods to a type — C++ member functions, Go methods. <code>&self</code> is a read-only receiver (<code>fn pos(&self) -> Cell</code>); <code>&mut self</code> would mutate.
<code>#[derive(Clone)]</code>	An attribute that auto-generates a trait impl (here, the field-by-field copy code).
<code>thread_local! +</code> <code>RefCell<T> +</code> <code>.borrow_mut()</code>	Interior mutability — worth understanding, because it is the only reason this bot can remember anything between turns. Rust forbids a plain mutable global (<code>static mut</code>) because a shared <code>&T</code> may never mutate. <code>thread_local!</code> gives a per-thread global; <code>RefCell<T></code> is a wrapper that moves the borrow check from compile time to <i>run</i> time, letting you take a temporary <code>&mut</code> via <code>.borrow_mut()</code> (or <code>&</code> via <code>.borrow()</code>) through a shared handle (it panics if you double-borrow). This is how <code>GE_LASTPOS</code> (the anti-stall streak) and <code>RH_RNG</code> (the random seed) persist across turns.
<code>.entry(k).or_insert(v)</code>	Map "insert-if-absent, return a mutable handle" — Python <code>dict.setdefault</code> . Initialises a troll's watchdog record on first sight.

A7. Numbers & small idioms

Construct	Meaning
<code>const</code>	Compile-time constant — C++ <code>constexpr</code> , Go <code>const</code> . <code>const TOTAL_TURNS: i32 = 300;</code>
<code>i32</code> vs <code>usize</code>	<code>i32</code> = 32-bit signed (coordinates, sizes). <code>usize</code> = pointer-sized unsigned, required for indexing any collection (like C++ <code>size_t</code>) — that is why the item indices <code>PLUM..WOOD</code> are <code>usize</code> and you see <code>as usize</code> / <code>as i32</code> casts between "count" and "coordinate".
turbofish <code>::<T></code>	Spell out a generic argument when inference cannot: <code>.collect::<vec<_>>()< code="">, <code>p[2].parse::<i32>()</code>. The <code>_</code> lets the compiler fill the rest.</vec<_>>()<></code>
<code>.max(1)</code> / <code>.min(k)</code>	Numeric clamp helpers. <code>u.movement_speed.max(1)</code> guards against divide-by-zero.
ceil-divide <code>(a+b-1)/b</code>	Integer round-up division with no floats. <code>(dist + ms - 1) / ms</code> = travel turns; <code>(health + chop - 1) / chop</code> = chops needed to fell.
<code>.saturating_add(1)</code>	Add without wrap-around (stops at the type's max). The watchdog streak counter is a <code>u8</code> .
<code>1 << 30</code> sentinel	A bit-shift producing a huge number used as "unreachable / infinity" in <code>min</code> comparisons — <code>unwrap_or(1 << 30)</code> .
<code>x.abs()</code>	Absolute value; builds Manhattan distance <code> dx + dy </code> .

Part B — The game and the strategy (what & why)

B1. The game in one paragraph

Two players share a small grid map over ~300 turns with a 50 ms budget each turn. You command "trolls" (workers) that `MOVE`, `HARVEST` fruit, `CHOP` /fell trees, `MINE` iron, `PLANT` a carried seed, `PICK` a banked item back out of your shack, `DROP` (bank) their load at the shack, or `TRAIN` a new troll. The map has fruit trees (**PLUM, LEMON, APPLE, BANANA**), **water** (which speeds nearby tree growth), and **iron**. A tree grows one size per cooldown tick, from size 1 up to **size 4 (MAX_SIZE)**; **only at size 4 does it start bearing fruit**. Felling a size-*s* tree yields *s* wood (capped by free carry capacity) and **removes the tree from the map** — so a half of the board can be permanently deforested.

B2. Why wood — the scoring economics

A player's score is $\Sigma(\text{plum} + \text{lemon} + \text{apple} + \text{banana}) + 4 \cdot \text{wood}$ banked in the shack; iron scores **0** (it is only a training currency). **Wood is leveraged 4x**, so a bot that funnels its effort into banked wood out-scores a fruit farmer of equal activity. Every design choice below follows from this one fact.

Species	Grow cooldown	Health (base + slope·size)	Why the bot cares
BANANA	6 (→4 next to water)	2 + 1·size — very low	Grows fastest <i>and</i> is cheapest to fell (~2 chops at size 2). The ideal crop. Useless for training, so every banana is pure surplus turned into wood.
APPLE	9 (→2 next to water)	8 + 3·size — tanky	Water-apple grows very fast but is slow to fell; used only as a <i>secondary</i> seed source when standing beside water.
PLUM / LEMON	8 (→3)	4 + 2·size	Training currency (a troll costs $n + \text{stat}^2$ of each fruit). Harvested only to fund the one chopper, then ignored.

B3. The strategy: two trolls, fixed roles ("gold-elite")

This bot replicates the decoded top-of-Gold meta. It commits to **exactly two trolls** (`GE_MAX_TROLLS = 2`) and assigns roles purely by the test `chop_power >= 2`:

Troll	Talents (ms, cc, hp, chop)	Job
Starter	the (1,1,1,1) troll you begin with	① harvest the fruit types the chopper still needs and MINE iron to <i>fund</i> it (roughly turns 20–77); ② once the chopper exists, run the banana printer (below); ③ if the farm is momentarily stocked, help chop (it has chop 1).
Chopper	(2, 2, 0, 2) — trained once	Perma-fell local trees and bank whenever it fills. <code>cc 2</code> means it is full after 2 wood → very short round-trips. <code>hp 0</code> (cannot harvest) makes it cheaper to train — it never needs to harvest.

Why not the boss's expensive `cc 4` chopper, or 3–4 trolls? Both were measured worse: a `cc 4` chopper's lemon cost ($n + 16$) drains the economy, and every alternative troll count / spec loses the head-to-head production race to the two-troll `(2,2,0,2)` configuration.

The **opening** ($\approx$ turns 1–77) funds and trains the single chopper. After that the game is one sustained loop: the starter keeps a dense cluster of fast-growing bananas alive next to the base, and the chopper fells them forever, banking ~40–82 wood by turn 300 ($\approx$ 160–330 points).

B4. The banana printer — the wood engine

The "printer" is a closed loop that keeps a cluster of bananas within `farm_r = 3` of the shack (cap `farm_max = 12`), so the chopper always has a ripe, low-health tree one step away — no travel, no idle.

- Seed acquisition:** the starter `PICK`s a banana out of the shack inventory (bananas get banked from felling/harvesting), or, if none are banked, `HARVEST`s a fruit-bearing banana (or water-apple) tree to obtain one seed.
- Planting:** carrying a banana, it walks to the nearest free base cell (water-adjacency is a mild tiebreak, since banana cooldown drops 6→4 beside water) and `PLANT`s it.
- Felling:** the chopper fells these local bananas, banks at `cc 2`, and the loop repeats. Banked fruit stays near zero — everything is converted into wood.

B5. ★ The v1.4.5 fix: a protected seed reserve (the crux)

The bug it fixes — the #1 arena loss. The chopper's normal rule is "fell any farm banana at size ≥ 2 ". But **a tree only bears fruit at size 4**. If the chopper fells *every* banana the instant it reaches size 2, no banana ever survives to size 4, so **none ever fruits**. Fruit is the only new source of banana seeds, so the seed supply only drains. When the last banked seed is spent, the printer has nothing to plant, our half of the map deforests, and — with no tree to fell and nothing to harvest — **both trolls park next to the shack and do nothing for the rest of the game**. That is the "shutdown" loss the arena kept handing us: ~5 plants left, chopper wandering, ~22 wood banked, a ~115-point defeat.

The fix — a seed reserve. Before choosing targets each turn, the bot marks the $K = 2$ (`GE_SEED_RESERVE`) *most-mature* farm bananas as a protected reserve the chopper is forbidden to fell. Those two bananas are allowed to ripen to size 4, where they **bear fruit**; the starter `HARVEST` s that fruit as fresh seeds and replants — closing the loop into a genuinely **self-sustaining farm**. Every *other* farm banana is still felled aggressively for wood, so throughput barely drops. The reserve is dropped during end-game "liquidation" (there is no future to sustain, so fell everything).

Measured effect on arena-faithful sparse maps: banked wood rises from ~45 → ~82, idle turns collapse, and the farm survives to the end instead of dying mid-game. Because Gold rank is win-rate driven, converting these shutdown *losses* into competitive games is pure ranking gain — this was the single change that mattered most for v1.4.5.

B6. Robustness features (never waste a turn)

Mechanism	What it prevents
Anti-starvation / never-idle — a troll with no normal task fells the nearest reachable tree of size ≥ 1 <i>anywhere</i> on the map rather than parking.	Zero-value turns. Felling a size-1 tree for a single wood beats standing still; on banana-poor maps this alone turned losses into wins.
Anti-stall watchdog — if a troll issued a <code>MOVE</code> but its position has not changed for 2+ turns, it is self-blocked; the bot forces a random side-step to a free adjacent cell.	The classic self-block: a troll spamming an identical, blocked <code>MOVE</code> for dozens of turns behind its own teammate or crop. Historically the #1 loss cause.
Seed reserve (B5)	Farm deforestation → the both-trolls-parked shutdown.
Endgame banking — any carried load is banked once <code>turns_left ≤ eta_home + 1</code> .	Carried goods score 0 at $t = 300$; a chopper's un-banked wood is 4 points each, thrown away.
Reserved-cell set — a per-turn <code>HashSet</code> of cells already claimed.	Two of our own trolls targeting the same cell (the referee forbids co-occupation; it would waste a move).
Shack-adjacent DROP — bank from an orthogonally-adjacent cell, never the shack tile itself.	The shack tile is unwalkable; a troll must <code>DROP</code> from beside it.

Part C — The code, block by block

This walks `docs/best-bot-v1.4.5.rs` in source order. Each block notes its purpose, the Part-A idioms it leans on, and the Part-B intent it serves. Line numbers refer to the readable file.

C1. `fn main` and the I/O loop (lines ~837-end)

CodinGame speaks to the bot over stdin/stdout. `main` reads a one-time header (`width height`) and the static ASCII map, then loops once per turn: parse this turn's state, decide, print the semicolon-joined commands, flush.

```
read "width height"           // once
read `height` map rows → parse_grid // walkable / shacks / iron / water (static all game)
loop turns:
  parse_turn → State           // inventories, trees, trolls this turn
  debug_log (no-op unless DEBUG) // stderr sim-fidelity echo
  cmds = decide_elite(&state)   // ← the whole brain
  writeln!(out, "{}", cmds.join(";")) // e.g. "MSG v1.4.5...;MOVE 0 2 1"
```

Why the import reads `use std::io::{self, BufRead, Write}` — the `self`. That one line pulls in **three** things, and they play two different roles:

- **`self = the module std::io itself.`** It is what lets the code write the *path-qualified* constructors `io::stdin()`, `io::stdout()`, `io::BufReader::new(...)`, `io::BufWriter::new(...)`. Drop the `self` and the name `io` is simply not in scope — you would have to spell every call `std::io::stdin()` in full, or add a second line `use std::io;`. The `self` inside the braces is exactly that second line, folded in: "also bring the module named by the prefix into scope."
- **`BufRead` and `Write` are traits**, imported by *name*. Here is the Rust rule with no analogue in Go or Python: **a trait's methods are only callable when the trait is in scope by name** — a path prefix is not enough. `io::stdin().lock()` hands back a reader whose `.read_line()` / `.lines()` live on `BufRead`; `writeln!(out, ...)` and `out.flush()` are `Write` methods. Without these two names in scope, those calls would not compile even though the module path already is. (This is also why the parser's signature `reader: &mut impl BufRead` needs the name — see A6.)

So `{self, BufRead, Write}` says, in one tidy line: *the module* (to reach the `io::` constructors) **plus the two traits** (to call the read/write methods). The closest Python is `import io` (the module handle) **combined with** `from io import BufRead, Write` (the names) — Rust merges both into a single `use`.

- **Idioms:** `parse_grid` is a `match ch { '0' => ..., '.' => ... }` over map characters (A4). `read_line` returns `Option<String>` and `parse_turn` propagates end-of-stream with `?` (A4); field parsing uses `.parse().unwrap()`, trusting the referee's format (A4).
- The static sets (`walkable`, `shacks`, `iron_cells`, `water_cells`) are parsed once and `.clone()` d into each turn's `State` (A2) — the map never changes.

C2. State structs: `State`, `Troll`, `Tree` (lines ~53-106)

Plain records (A3) holding the parsed world. `Troll` and `Tree` carry small `impl` method blocks (A6):

- `Troll::pos()` → its `(x,y)`; `total_carried() = self.carry.iter().sum(); free_capacity() = carry_capacity - total_carried()`. These three drive nearly every "am I full / where am I" test.
- `carry: [i32; 6]` is indexed by the item constants `PLUM=0 ... WOOD=5` (all `usize`, A7) — e.g. `u.carry[BANANA]`.
- `#[derive(Clone)]` (A6) lets `decide_elite` take a working copy of the troll list (`state.my_trolls.clone()`) so it can sort it by id without touching the shared `State`.

C3. Configuration constants `GE_*` (lines 187-197)

The whole strategy is captured in eleven `const`s (A7). These are the swept, verified arena defaults.

Constant	Value	Meaning
<code>GE_SPEC</code>	(2,2,0,2)	the one chopper's talents (ms, cc, hp, chop)
<code>GE_MAX_TROLLS</code>	2	total troll cap — never train a third
<code>GE_FARM_R</code>	3	banana-farm radius around the shack
<code>GE_FARM_MAX</code>	12	max base trees the printer maintains
<code>GE_FELL_SIZE</code>	2	min size the chopper fells (non-farm trees)
<code>GE_FARM_FELL</code>	2	min size to fell a <i>non-reserved</i> farm banana
<code>GE_CHOP_R</code>	10	max shack-distance the chopper roams for a normal fell
<code>GE_LIQ_T</code>	34	final turns: liquidation — fell anything reachable
<code>GE_SEED_RESERVE</code>	2	K protected most-mature bananas (B5)
<code>GE_STARTER_CHOP</code>	true	let the chop-1 starter help fell when the farm is stocked
<code>GE_MIN_TURNS_LEFT</code>	20	no training inside the last 20 turns

C4. Per-turn setup & the training gate (inside `decide_elite`, lines ~222-255)

At turn 1 the persistent maps are cleared. Then the bot snapshots the shacks, inventory and `turns_rem`, sorts its trolls by id, and decides *whether to train the chopper this turn*:

```
let want_chopper = n < GE_MAX_TROLLS && !my.iter().any(|u| u.chop_power >= 2);
let cost         = training_cost(n, spec);           // n + ms², n + cc², n + hp², n + chop²
(iron)
let train_now    = want_chopper && mb_afford(inv, &cost, have_iron);
let need_iron    =
    have_iron && want_chopper && inv[IRON] < cost[IRON] && afford_fruit_only(inv, &cost);
let need_fund: [bool; 3] = [inv[0] < cost[0], inv[1] < cost[1], inv[2] < cost[2]];
```

- **Idioms:** `.iter().any(...)` (A5) tests "do we already have a chopper?"; the tuple `spec` is `GE_SPEC` (A3); indexing `inv[IRON]` uses a `usize` constant (A7).
- **Intent (B3):** `want_chopper` is true until one `(_,_,_,≥2)` troll exists. `need_fund[t]` is the per-fruit deficit that steers the starter's harvesting; `need_iron` flags "fruit is ready but iron isn't → go mine".

C5. ★ The seed reserve & the `fell_ok` gate (lines ~263-283)

This is the v1.4.5 change (B5) and the heart of the file. First, collect the farm's bananas, sort them *most-mature-first*, and reserve the top `K`. Then `fell_ok` — a closure every targeting step consults — refuses to fell a reserved cell.

```
let mut seed_cells: HashSet<Cell> = HashSet::new();
if GE_SEED_RESERVE > 0 && !liquidation {
    let mut fb: Vec<&Tree> = state.trees.iter()
        .filter(|p| p.tree_type == "BANANA" && manhattan(p.pos(), shack) <= farm_r)
        .collect();
    fb.sort_by_key(|p| (-p.size, -p.fruits, manhattan(p.pos(), shack), p.pos())); // mature
    first
    for p in fb.into_iter().take(GE_SEED_RESERVE) { seed_cells.insert(p.pos()); }
}
let fell_ok = |p: &Tree| -> bool {
    if seed_cells.contains(&p.pos()) { return false; } // protected seed source
    if liquidation { return p.size >= 1; } // endgame: fell anything
    let farm_banana = p.tree_type == "BANANA" && manhattan(p.pos(), shack) <= farm_r;
    p.size >= if farm_banana { GE_FARM_FELL } else { fell_size }
};
```

- **Idioms:** a whole iterator pipeline — `.iter().filter(...).collect()` into a `Vec<&Tree>` of borrows (A5, A2); `sort_by_key` with a **tuple key** for lexicographic "size desc, then fruits desc, then nearest" (A5); `.into_iter().take(K)` to keep the top `K` (A5). `fell_ok` is a closure capturing `seed_cells`, `liquidation`, `farm_r`, `fell_size` (A5); early `return`s make it read like a guard cascade.
- **Intent (B5):** "most mature first" guarantees the two bananas closest to fruiting are the ones spared, so the farm reaches size 4 and re-seeds itself.

C6. Target selection: `nearest_fell` (lines ~337-354)

A representative iterator chain: among all trees, keep the fellable, own-half, in-range, reachable, un-reserved ones, and return the position of the one that is *fastest to fell* (travel turns + chop turns).

```
let nearest_fell = |free_needed: bool| -> Option<Cell> {
    if free_needed && u.free_capacity() == 0 { return None; }
    state.trees.iter()
        .filter(|p| fell_ok(p)) // C5 gate (respects the
    reserve)
        .filter(|p| own_half(p) && within_roam(p))
        .filter(|p| d.contains_key(&p.pos()) && !reserved.contains(&p.pos()))
        .min_by_key(|p| { // pick the cheapest
            let steps = (d[&p.pos()] + u.movement_speed - 1) / u.movement_speed.max(1);
            let chop_t = (p.health + u.chop_power.max(1) - 1) / u.chop_power.max(1);
            steps + chop_t // ceil-divides (A7)
        })
        .map(|p| p.pos()) // Option<&Tree> ->
    Option<Cell>
};
```

- **Idioms:** three `.filters` then `.min_by_key` returning an `Option` (A4, A5); the key uses two ceil-divisions and `.max(1)` divide-by-zero guards (A7); a trailing `.map` converts the winning `&Tree` into its `Cell`. `d` is this troll's BFS distance map (`bfs_distances` from the shack-less source = the troll's own cell).

- **Intent (B3/B4):** "closest and cheapest to fell" — banana health 2+size is tiny, so a ripe farm banana almost always wins, keeping the chopper on the local loop.

C7. The chopper branch (lines ~356-410)

For the `chop_power ≥ 2` troll, a fixed priority cascade, each arm ending in `continue`:

CHOPPER:

- | | | |
|---|---------------------------|------------------|
| 1. carrying a load & endgame (<code>turns_left ≤ eta_home+1</code>) | → bank it | // B6 endgame |
| 2. full (<code>free_capacity == 0</code>) | → bank | |
| 3. standing on a <code>fell_ok</code> tree | → CHOP (reserve the cell) | |
| 4. nearest_fell(false) exists | → MOVE toward it | |
| 5. ★ ANTI-STARVATION: nearest reachable <code>size≥1</code> tree | → CHOP / MOVE | // B6 never-idle |
| 6. carrying something → bank the partial load; else | → park by the shack | |

Idioms: `state.trees.iter().find(|p| p.pos() == u.pos())` (A5) is "am I standing on a tree?"; writing to the `GE_MEM` thread-local via `.with(|m| m.borrow_mut().insert(...))` (A6) records the target. Step 5 is a second, *ungated* `min_by_key` over all `size≥1` trees — deliberately ignoring `fell_ok/own-half` so the chopper always has something to do.

C8. The starter branch (lines ~411-585)

For the non-chopper, a six-step cascade that *is* the printer plus funding:

STARTER:

1. endgame banking (same rule)
2. carrying a banana & farm has room → walk to a free base cell & PLANT // printer: plant
3. full → bank
4. standing on a ripe tree we want → HARVEST
(funding phase: a deficit fruit type; after funding: only banana / water-apple seeds)
5. FUNDING: MINE iron / go harvest the fruit types the chopper still needs (`need_fund`)
6. BANANA PRINTER: PICK a banked banana at the shack → else HARVEST a native banana for a seed
(then, if the farm is full / no seeds: help CHOP, anti-starvation, or park)

```
// 5) BANANA PRINTER: keep the farm stocked with bananas
if base_trees < farm_cap {
    if manhattan(u.pos(), shack) == 1 && inv[BANANA] > 0 && u.free_capacity() > 0 {
        cmd_by_id.insert(u.id, format!("PICK {} BANANA", u.id)); continue; // fastest seed
    }
    if inv[BANANA] > 0 { cmd_by_id.insert(u.id, park_cmd(u, &d)); continue; } // step to the
    shack to PICK
    // no banked seeds: harvest a native banana (or water-apple) for one
    let seed_tree = state.trees.iter()
        .filter(|p| p.fruits > 0 && d.contains_key(&p.pos())) &&
        !reserved.contains(&p.pos()))
        .filter(|p| p.tree_type == "BANANA"
            || (p.tree_type == "APPLE" && state.water_cells.iter().any(|w| manhattan(*w,
p.pos()) == 1)))
        .min_by_key(|p| d[&p.pos()]).map(|p| p.pos());
    // ... if Some(tc): MOVE toward it ...
}
```

Intent (B4/B5): the fruit that step 6 harvests comes precisely from the **size-4 reserved bananas** of C5 — that is the loop closing. During funding, step 4/5 instead chase `need_fund` fruit and iron; `ge_fruit_ty` (a `match`, A4) maps a tree type name to its inventory index.

C9. Endgame banking & liquidation

Two time-based overrides woven through the cascades: (1) **endgame banking** — computed per troll as `eta = ceil(dist_home / ms) + 1`; if `turns_rem ≤ eta + 1` the troll banks whatever it holds (B6), because carried goods score 0 at `t=300`. (2) **Liquidation** — once `turns_rem ≤ GE_LIQ_T` (34), `fell_ok` returns `size ≥ 1` for *any* tree and the seed reserve is skipped: with no future to sustain, every standing tree is just un-banked wood to be cut.

C10. The anti-stall watchdog (lines ~589-635)

After the cascades fill `cmd_by_id`, one pass detects self-blocked trolls (B6) using the persistent `GE_LASTPOS` map:

```
GE_LASTPOS.with(|cell| {
    let mut m = cell.borrow_mut(); // interior mutability (A6)
    for t in &my {
        let cur = t.pos();
        let is_move = cmd_by_id.get(&t.id).map_or(false, |c| c.starts_with("MOVE "));
        let entry = m.entry(t.id).or_insert((cur.0, cur.1, 0u8)); // A6 entry API
        let stuck = entry.0 == cur.0 && entry.1 == cur.1;
        entry.2 = if stuck && is_move { entry.2.saturating_add(1) } else { 0 };
        // ... if streak ≥ 2 & still MOVE-ing: pick a random free neighbour and override the
    command ...
    }
});
```

Idioms: `.with(|cell| ...)` is how you access a `thread_local!`; `.borrow_mut()` takes the run-time-checked `&mut` (A6); `.entry().or_insert()` initialises the record (A6); the streak is a `u8` bumped with `.saturating_add(1)` (A7); tuple fields `entry.0/.1/.2` hold `(x, y, streak)` (A3). The random side-step cell is chosen with `rh_rand()` (the xorshift over `RH_RNG`).

C11. Command emission

Finally the per-id commands are gathered and printed deterministically: sort the troll ids, push each command in id order; prepend `MSG v1.4.5-seedreserve` on turn 1; append `TRAIN 2 2 0 2` if `train_now` holds, we are not inside the last 20 turns, and the shack tile is free; emit `WAIT` if somehow empty. `main` joins the vector with `;` into the single output line the referee expects.

Appendix — where things live

File	What
<code>docs/best-bot-v1.4.5.rs</code>	the readable single-file bot documented here (submission with dead code stripped; compiles with <code>rustc --edition 2021 -O</code>)
<code>cgauto/submissions/v1.4.5-seedreserve.rs</code>	the exact arena submission (identical live path, plus unused experimental code)
<code>rust/src/strategies/gold_elite.rs</code>	the same algorithm as a heavily-commented <code>impl Strategy for GoldElite</code> — the clearest place to read the intent

Generated 2026-07-04. The live decision function is `fn decide_elite(state: &State) -> Vec<String>;` the game is CodinGame Spring Challenge 2026, "Troll Farm".